

B U P T

密码工程

第4讲、若干基础问题（2） 随机数产生、素数与大数运算

授课教师：王晨宇

时间：2026年3月26日



目录

contents

B U P T



3.3 随机数产生

3.4 素数与大数运算

3.3

随机数产生



随机数产生



✓ 3.3.1 随机性介绍

✓ 3.3.6 真随机序列发生器

✓ 3.3.2 线性同余发生器

✓ 3.3.7 提取随机性

✓ 3.3.3 反馈移位寄存器

✓ 3.3.4 Geffe发生器

✓ 3.3.5 附加式发生器



随机数产生

3.3.1 随机性介绍

- **目的：**产生好的随机性在许多密码运算中关键的一步，因此我们需要使用随机数发生器(RNG)来产生密钥。
- **随机数发生器种类：**真随机数发生器(TRNG)、伪随机数发生器(PRNG)、密码学伪随机数发生器(CSPRNG)
- TRNG生成的是真正的随机数，PRNG生成的是看起来随机的随机数。



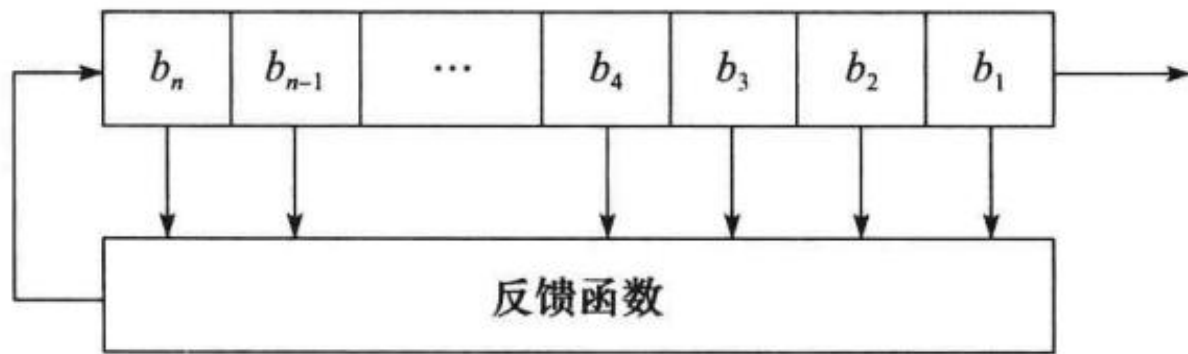


- 什么是线性同余发生器？ $X_n = (aX_{n-1} + b) \bmod m$ 形式的伪随机序列发生器。
- 其中 X_n 是序列的第 n 个数， X_{n-1} 是序列的第 $n-1$ 个数， 变量 a, b, m 是常数， a 是乘数， b 是增量， m 是模。密钥， 即种子， 是初始值 X_0 。
- 这种发生器的初始值不会超过 m 。如果 a, b, m 都是可选的， 那么发生器将是一个最大周期发生器， 周期为 m 。在选择常数时需要仔细， 以保证找到最大的周期， 右表给出了一些线性同余发生器好的常数。

溢出位置	a	b	m	溢出位置	a	b	m
2^{20}	106	1 283	6 075	2^{28}	1 277	24 749	117 128
2^{21}	211	1 663	7 875		741	66 037	312 500
2^{22}	421	1 663	7 875		2 041	25 673	121 500
2^{23}	430	2 531	11 979	2^{29}	2 311	25 367	120 050
	936	1 399	6 655		1 807	45 289	214 326
	1 366	1 283	6 075		1 597	51 749	244 944
2^{24}	171	11 213	53 125		1 861	49 297	233 280
	859	2 531	11 979		2 661	36 979	175 000
	419	6 173	29 282		4 081	25 673	121 500
	967	3 041	14 406		3 661	30 809	145 800
2^{25}	141	28 411	134 456	2^{30}	3 877	29 573	139 968
	625	6 571	31 104		3 613	45 289	214 326
	1 541	2 957	14 000		1 366	150 889	714 025
	1 741	2 731	12 960	2^{31}	8 121	28 411	134 456
	1 291	4 621	21 870		4 561	51 349	243 000
	205	29 573	139 968		7 141	54 773	259 200
2^{26}	421	17 117	81 000	2^{32}	9 301	49 297	233 280
	1 255	6 173	29 282		4 096	150 889	714 025
	281	28 411	134 456	2^{33}	2 416	374 441	1 771 875
2^{27}	1 093	18 257	86 436	2^{34}	17 221	107 839	510 300
	421	54 773	259 200		36 261	66 037	312 500
	1 021	24 631	116 640	29^{35}	84 589	45 989	217 728
	1 021	25 673	121 500				



- 自从电子时代开始以来，基于移位寄存器的序列密码已经广泛地用于军事密码学中。
- **反馈移位寄存器：**由移位寄存器和反馈函数组成。移位寄存器是一个位序列，每次需要一位，移位寄存器中所有位右移一位。新的最左端的位根据寄存器中其他位计算得到。移位寄存器的输出常常是最低有效位。移位寄存器的周期是指输出序列从开始到重复时的长度。
- 密码设计者喜欢用移位寄存器构造序列密码，因为它容易通过数字硬件实现。



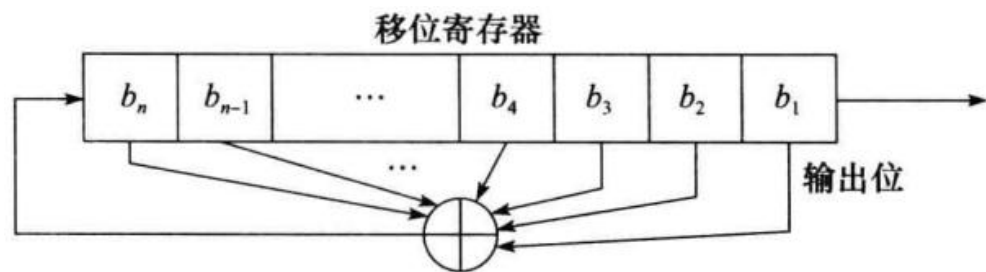


随机数产生

3.3.3 线性反馈移位寄存器(LFSR)

B U P T

- 最简单的反馈移位寄存器是**线性反馈移位寄存器（LFSR）**。
- 反馈函数与寄存器中的某些位简单异或，这些位叫做抽头序列，有时也叫做Fibonacci配置。
- 一个 n 级的LFSR最多存储 $2^n - 1$ 种状态(减一是因为去掉了全0序列，LFSR种只有0的话，输出会一直是0，是不可用的)。例如一个3级LFSR最多遍历001, 010, 011, 100, 101, 110, 111共7种状态。
- LFSR及基于LFSR的随机序列发生器中使用了电子逻辑标识符。在文本中， $\oplus$ 表示异或， $\wedge$ 表示与， $\vee$ 表示或， $\neg$ 表示非



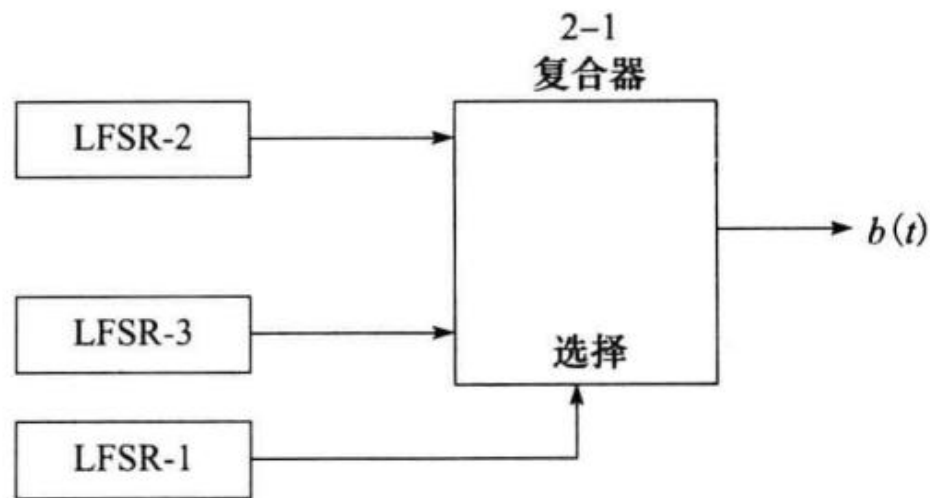


随机数产生

3.3.4 Geffe发生器

B U P T

- Geffe发生器使用了3个LFSR，它们以非线性方式组成，两个LFSR作为复合器的输入，第三个LFSR控制复合器的输出。
- 如果 a_1, a_2, a_3 是三个LFSR的输出，则Geffe发生器的输出表示为 $b = (a_1 \wedge a_2) \oplus ((\neg a_1) \wedge a_3)$ ，可以看出LFSR-1主要起选择作用，当 $a_1 = 1$ 时， $b = a_2$ ，输出LFSR-2的序列；当 $a_1 = 0$ 时， $b = a_3$ ，输出LFSR-3的序列
- 这个发生器的周期为是三个LFSR周期的最小公倍数。





- Geffe发生器从理论上看起来很好，但实质上很弱，并不能抵抗相关攻击。
- Geffe发生器的输出与LFSR-2的输出有75%的时间是相同的。因此如果已知反馈抽头，便能猜出LFSR-2的初值和寄存器所产生的输出序列。然后就能计算出LFSR-2的输出与这个发生器的输出相同的次数。如果猜错了，两个序列相同的概率为50%；如果猜对了，两个序列相同的概率为75%。
- 类似地，发生器的输出与LFSR-3的输出相等的概率为75%。有了这种相关性，密钥序列发生器很容易被破译。



- 附加式发生器用随机字取代了随机位。发生器的初始状态是一个 n 位字：8位、16位或32位，无论是哪一种，令其为 $X_1, X_2, \dots, X_m$ 。初始状态就是密钥。发生器的第 i 个字是： $X_i = (X_{i-a} + X_{i-b} + \dots + X_{i-m}) \bmod 2^n$ 。
- 如果系数 $a, b, c, \dots, m$ 选择正确，那么发生器的周期至少是 $2^n - 1$ 。
- Fish发生器是一种基于收缩式发生器的附加式发生器。它产生一个32位字的密钥序列，这个密钥序列与明文序列异或产生密文序列，或者与密文序列异或产生明文序列。



随机数产生 3.3.5 Fish发生器

□ 首先，使用这两个附加式发生器。密钥是这些发生器的初始值。

$$A_i = (A_{i-55} + A_{i-24}) \bmod 2^{32}, \quad B_i = (A_{i-52} + A_{i-19}) \bmod 2^{32}$$

□ 这些序列都是收缩式的，作为一个组合处理，依赖于 B_i 的最低有效位：如果它是1，则利用组合；如果它是0，则忽略组合。 C_j 是从 A_i 中产生的序列，而 D_j 是从 B_i 中产生的序列。这些字用在组合中 $(C_{2j}, C_{2j+1}, D_{2j}, D_{2j+1})$ 产生两个32位输出字： K_{2j}, K_{2j+1}

$$\square E_{2j} = C_{2j} \oplus (D_{2j} \wedge D_{2j+1}), F_{2j} = C_{2j+1} \wedge (E_{2j} \wedge C_{2j+1}), K_{2j} = E_{2j} \oplus F_{2j}, K_{2j+1} = C_{2j+1} \oplus F_{2j}.$$

□ 这个算法运行速度很快。在33MHz的486机器上，Fish加密数据的C语言实现速度为15Mb/s。但不幸的是，它仍然是不可靠的，一次攻击大概需要 2^{40} 次运算

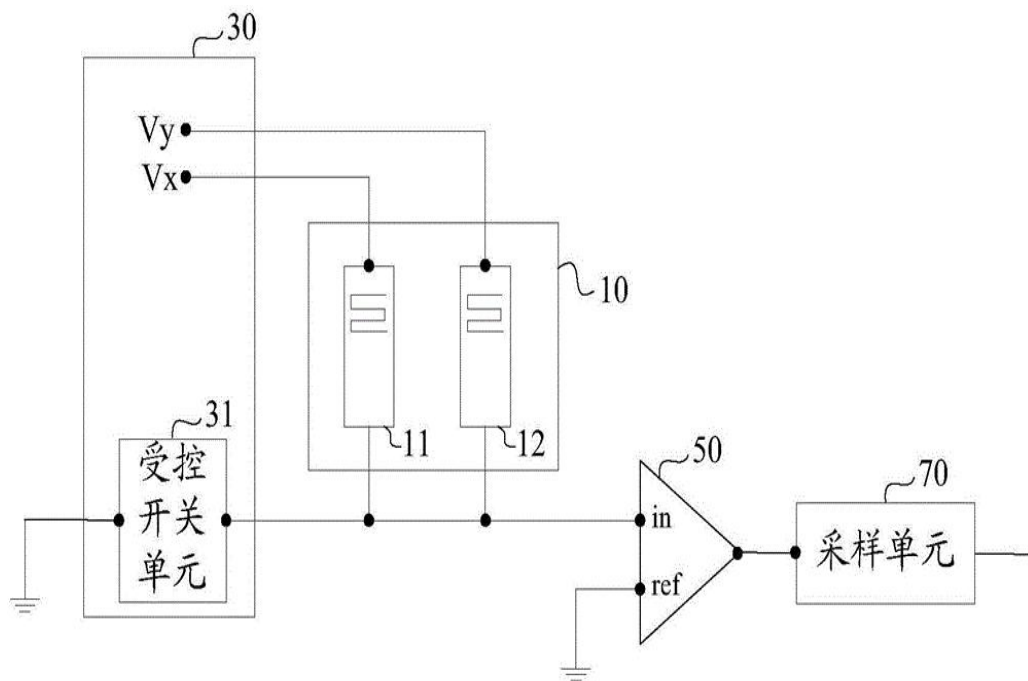


随机数产生

3.3.6 真随机序列发生器

B U P T

- 有时密码学意义上安全的伪随机数并不足够好。在密码学多个应用中需要的是真正的随机数。
- 伪随机序列发生器为什么不够好？如果敌方得到了一份此发生器和主密钥的副本，他们就能产生相同的密钥来破解密码系统。
- 一个真正的随机序列发生器产生的序列是不可再现的，任何人即使是自己都不能再次产生。
- 是否能产生真正的随机数，长期以来这个问题都处在激烈的争论之中。





- **使用随机噪声：**采集大量随机数的最好方法是选取真实世界的自然随机性，这种方法经常需要一个特定的硬件，但仍需要一定的计算机技巧。
- **使用计算机时钟：**如果想用一个随机位(或者是多个)，那么从任何时钟寄存器取最低有效位即可。因为UNIX系统的用户空间程序无法直接读写硬件寄存器（如CPU的时间戳计数器）。这些操作属于特权指令，仅允许内核或特定驱动程序执行。若用户程序强行尝试，会触发段错误（Segmentation Fault）或被操作系统拦截。所以在UNIX系统中是不可行的，但它在Windows上是可行的。
- **测量键盘反应时间：**测量连续按键的时间，然后取这些测量的最低有效位。这个方法在UNIX终端可能行不通，因为UNIX系统通常被设计用于多用户和多任务环境，键盘输入的处理可能会受到其他任务的干扰，导致反应时间不稳定，而Windows拥有成熟的第三方工具（如KeyboardTest），通过系统API直接获取键盘时间戳，可精确测量按键按下到系统响应的延迟。



□产生随机数的最好办法就是找出许多似乎是随机的事件，然后从中提取随机性。这种随机性能存储在一个应用程序需要的库或存储器中。单向散列函数即可用于此，将能找到的事件进行散列运算，可尝试：

- 每个按键的一个副本
- 鼠标命令
- 扇区数、一天的某个时间和寻找每个磁盘操作的延迟
- 实际鼠标位置
- 显示器扫描线数
- 实际显示图像的内容
- CPU负载



1.使用真随机数据就一定没有问题吗？如果有，使用真随机数据存在什么问题？

3.4

素数与大数运算



素数与大数运算



✓ 3.4.1 素数与最大公因子

✓ 3.4.2 求模逆元

✓ 3.4.3 欧拉函数

✓ 3.4.4 中国剩余定理

✓ 3.4.5 勒让德符号、雅可比符号

✓ 3.4.6 伽罗瓦域中的计算

✓ 3.4.7 因子分解

✓ 3.4.8 素数的产生



- **什么是素数?** 比1大, 其因子只有1和它本身, 没有其他数可以整除它。素数是无限的, 密码学(尤其是公钥密码学)常用大素数。
- 两个数互素是指当他们除了1意外没有共同的因子。换句话说, 如果 a 和 n 的最大公因子等于1, 那么可写作 $\gcd(a, n) = 1$ 。
- 计算两个数的最大公因子最容易的方法就是用欧几里得算法。它是幸存到现在最古老的算法, 至今它仍是完好且常用的。
- **欧几里得算法又称辗转相除法**, 是指用于计算两个正整数 a, b 的最大公约数。计算公式为: $\gcd(a, b) = \gcd(b, a \bmod b)$ 。
- 欧几里得算法基于如下原理: 两个整数的最大公约数等于其中较小的数和两数相除的余数的最大公约数。



□ 欧几里得算法的C语言描述如下：

```
/* returns gcd of x and y */
int gcd (int x, int y)
{
    int g;
    if (x < 0)
        x = -x;
    if (y < 0)
        y = -y;
    if (x + y == 0)
        ERROR;
    g = y;
    while (x > 0) {
        g = x;
        x = y % x;
        y = g;
    }
    return g;
}
```



- **什么是逆元?** 4的乘法逆元是 $1/4$ ，因为 $4 \times 1/4 = 1$ 。在模运算的领域这个问题更复杂： $4 \times x \equiv 1 \pmod{7}$ ，寻找一组 x 和 k 使得： $4x = 7k + 1$ 。
- 更为一般的问题是寻找一个 x ，使得 $1 = (a \times x) \pmod{n}$ ，也可写作 $a^{-1} \equiv x \pmod{n}$ 。
- 解决模的逆元问题很困难，有时候有一个方案，有时候没有。例如，5模14的逆元是3，2模14却没有逆元。
- 一般而言，如果 a 和 n 是互素的，那么 $a^{-1} \equiv x \pmod{n}$ 有唯一解；如果 a 和 n 不是互素的，那么 $a^{-1} \equiv x \pmod{n}$ 没有解。如果 n 是素数，那么从 $1 \sim n - 1$ 的每一个数与 n 都是互素的，且在这个范围内恰好有一个逆元。
- **如何计算求模逆元?** 扩展欧几里得算法。



- 裴蜀定理：对于任意的整数 a, b ，都存在一对整数 x, y 使得 $ax + by = \gcd(a, b)$ 成立。
- 对于 $ax + by = m$ 这个不定方程，如果存在一组合法的解 (x, y) ，那么一定会有 $\gcd(a, b) \mid m$ ，即 m 是 $\gcd(a, b)$ 的倍数，不仅要知道有没有解，还要知道解是什么。
- 扩展欧几里得算法是一种求解不定方程的方法。算法的实现基于裴蜀定理的证明。实质上是相当于做欧几里得算法的时候对不定方程 $ax + by = m$ 的 x, y 也做了更改。所以经过扩展欧几里得算法处理过的 x, y 就已经是一组合法的可行解了。



□ 算法C语言实现如下:

```
int exgcd(int a,int b,int &x,int &y)
{
    if(b==0)
    {
        x=1,y=0;
        return a;
    }
    int d=exgcd(b,a%b,x,y);
    int k=x;
    x=y;
    y=k-a/b*y;
    return d;
}
```



- **费马小定理**：如果 m 是一个素数，且 a 不是 m 的倍数，那么有： $a^{m-1} \equiv 1 \pmod{m}$
- 欧拉函数也被称作欧拉 ϕ 函数，写作 $\phi(n)$ ，它表示模 n 的余数化简集中元素的数目。换句话说， $\phi(n)$ 表示与 n 互素的小于 n 的正整数的数目($n > 1$)。
- 如果 n 是素数，那么 $\phi(n) = n - 1$ ；如果 $n = pq$ ，且 p 和 q 互素，那么 $\phi(n) = (p - 1)(q - 1)$
- 根据费马小定理的欧拉推广，如果 $\gcd(a, n) = 1$ ，那么 $a^{\phi(n)} \pmod{n} = 1$ ，现在计算 a 模 n 很容易： $x = a^{\phi(n)-1} \pmod{n}$ 。
- 例：5模7的逆元是多少？ $\phi(7) = 6$ 。 $5^{6-1} \pmod{7} = 3$



- 如果已知 n 的素因子，那么就能用**中国剩余定理**求解整个方程组。这个定理的最初形式是由1世纪的中国数学家孙子发现的。
- 一般而言，如果 n 的素因子可分解为 $n = p_1 \times p_2 \times \dots \times p_t$ ，那么方程组 $(x \bmod p_i) = a_i, i = 1, 2, \dots, t$ 有唯一解，这里 $x < n$ (注意有些素数可能不止一次出现，例如可能 $p_1 = p_2$)。换句话说，一个数(小于一些素数之积)被它的余数模这些素数唯一确定。
- 例：取素数3和5，取一个数14，那么 $14 \bmod 3 = 2$ ， $14 \bmod 5 = 4$ 。则小于 $3 \times 5 = 15$ 且具有上述余数的数只有14，即由这两个余数唯一地确定了数14。



- 如果对任意 $a < p$ 和 $b < q$ (p 和 q 均为素数), 那么当 $x < p \times q$ 时, 存在一个唯一的 x , 使得 $x \equiv a \pmod{p}$ 且 $x \equiv b \pmod{q}$
- 为求出这个 x , 首先用**欧几里得算法**找到 u , 使得 $u \times q \equiv 1 \pmod{p}$, 然后计算
$$x = \left(((a - b) \times u) \pmod{p} \right) \times q + b。$$
- 中国剩余定理的一个推论可以求出类似问题的解: 如果 p 和 q 均为素数, 且 $p < q$, 那么存在唯一的一个 $x < p \times q$, 使得 $a \equiv x \pmod{p}$ 且 $b \equiv x \pmod{q}$
- 如果 $a \geq b \pmod{p}$, 则 $x = \left(((a - (b \pmod{p})) \times u) \pmod{p} \right) \times q + b$; 如果 $a < b \pmod{p}$, 那么 $x = \left(((a + p - (b \pmod{p})) \times u) \pmod{p} \right) \times q + b$



□ 用C语言写的中国剩余定理如下:

```
/* r is the number of elements in arrays m and u;
m is the array of (pairwise relatively prime) moduli
u is the array of coefficients
return value is n such than n == u[k]%m[k] (k=0..r-1) and
    n < m[0]*m[1]*...*m[r-1]
*/

/* totient() is left as an exercise to the reader. */

int chinese_remainder (size_t r, int *m, int *u)
{
    size_t i;
    int modulus;
    int n;
    modulus = 1;
    for (i=0; i<r; ++i)
        modulus *= m[i];
    n = 0;
    for (i=0; i<r; ++i) {
        n += u[i] * modexp(modulus / m[i], totient(m[i]),
            m[i]);
        n %= modulus;
    }

    return n;
}
```



- 如果 p 是素数, 且 $a < p$, 如果 $x^2 \equiv a \pmod{p}$ 对某些 x 成立, 那么称 a 是对模 p 的二次剩余。
- 如果 a 是对模 n 的二次剩余, 那么它必定是对模 n 的所有素因子的二次剩余。
- 例如: $n=15$, 其素因子为3和5。取 $a=4$, 因为 $169 = 13^2 \equiv 4 \pmod{15}$, 所以4是模15的二次剩余, $9 = 3^2 \equiv 4 \pmod{5}$, $4 = 2^2 \equiv 1 \pmod{3}$, 所以4是模3和模5的二次剩余。
- 当 p 是奇数, 对模 p 的二次剩余数目恰好是 $(p-1)/2$, 且与非二次剩余的数目相同。
- 如果 n 是两个素数 p 和 q 的积, 那么模 n 恰好有 $(p-1)(q-1)/4$ 个二次剩余。



□ 勒让德符号写作 $L(a, p)$, 当 a 为任意整数且 p 是一个大于2的素数时, 它等于0、1或-1。

$L(a, p) = 0$ 如果 a 被 p 整除

$L(a, p) = 1$ 如果 a 是二次剩余

$L(a, p) = -1$ 如果 a 是非二次剩余

□ 一种计算 $L(a, p)$ 的方法是: $L(a, p) = a^{(p-1)/2} \bmod p$, 或者用下面的方法计算:

(1) 如果 $a=1$, 那么 $L(a, p)=1$ 。

(2) 如果 a 是偶数, 那么 $L(a, p) = L(a/2, p) \times (-1)^{(p^2-1)/8}$ 。

(3) 如果 a 是奇数 (且 $a \neq 1$), 那么 $L(a, p) = L(p \bmod a, a) \times (-1)^{(a-1) \times (p-1)/4}$ 。

注意, 这也是确定 a 是否是对模 p 的二次剩余的有效方法 (虽然仅当 p 是素数时成立)。



□雅可比符号写作 $J(a, n)$ ，是勒让德符号的合数模的一般化表示，它定义在任意整数 a 和奇整数 n 上。这个函数首先出现在素数测试中。雅可比符号是基于 n 的除数的余数化简集上的函数，可按多种方法进行运算，下面列举一种方法。

□首先是一些定义：

定义 1: $J(a, n)$ 只定义在 n 为奇数的情况下。

定义 2: $J(0, n) = 0$ 。

定义 3: 如果 n 是素数，且 n 能整除以 a ，那么 $J(a, n) = 0$ 。

定义 4: 如果 n 是素数，且 a 是模 n 的一个二次剩余，那么 $J(a, n) = 1$ 。

定义 5: 如果 n 是素数，且 a 是模 n 的一个非二次剩余，那么 $J(a, n) = -1$ 。

定义 6: 如果 n 是合数，那么 $J(a, n) = J(a, p_1) \times \cdots \times J(a, p_m)$ ，其中 $p_1 \cdots p_m$ 是 n 的素因子。



□ 计算雅可比符号的递归算法如下：

规则 1: $J(1, n) = 1$ 。

规则 2: $J(a \times b, n) = J(a, n) \times J(b, n)$ 。

规则 3: 如果 $(n^2 - 1)/8$ 是偶数, 则 $J(2, n) = 1$; 否则, 为 -1 。

规则 4: $J(a, n) = J((a \bmod n), n)$ 。

规则 5: $J(a, b_1 \times b_2) = J(a, b_1) \times J(a, b_2)$ 。

规则 6: 如果 a 和 b 都是奇数, 且它们的最大公因子是 1, 那么

规则 6a: 如果 $(a-1)(b-1)/4$ 是偶数, 那么 $J(a, b) = J(b, a)$ 。

规则 6b: 如果 $(a-1)(b-1)/4$ 是奇数, 那么 $J(a, b) = -J(b, a)$ 。

□ 如果已知 n 是素数, 则不需要运行以上算法, 而只需简单地计算 $a^{((n-1)/2)} \bmod n$ 。

这种情况下, 雅可比符号等同于勒让德符号。

□ 雅可比符号不能用来确定一个数 a 是否是对模 n 的二次剩余 (除非 n 是素数)。



□雅可比符号计算算法的C语言实现如下：

```
/* This algorithm computes the Jacobi symbol recursively */
int jacobi(int a, int b)
{
    int g;
    assert(odd(b));
    if (a >= b) a %= b;          /* by Rule 4 */
    if (a == 0) return 0;        /* by Definition 2 */
    if (a == 1) return 1;        /* by Rule 1 */
    if (a < 0)
        if ((b-1)/2 % 2 == 0)
            return jacobi(-a,b);
        else
            return -jacobi(-a,b);
    if (a % 2 == 0) /* a is even */
        if ((b*b - 1)/8 % 2 == 0)
            return +jacobi(a/2, b)
        else
            return -jacobi(a/2, b) /* by Rule 3 and Rule 2 */
    g = gcd(a,b);
    assert(odd(a)); /* this is guaranteed by the (a % 2 == 0)
test */
    if (g == a) /* a exactly divides b */
        return 0; /* by Rules 5 and 4, and Definition 2 */
    else if (g != 1)
        return jacobi(g,b) * jacobi(a/g, b); /* by Rule 2 */
    else if (((a-1)*(b-1)/4) % 2 == 0)
        return +jacobi(b,a); /* by Rule 6a */
    else
        return -jacobi(b,a); /* by Rule 6b */
}
```



- 如果 p 是一个素数或一个大素数的幂，那么就存在数学家称之为有限域的东西，也就是伽罗瓦域，用 $GF(p)$ 表示。
- 在伽罗瓦域上，非零元的加减乘除均有定义。有一个加法单位元0和一个乘法单位元1。每一个非零数都有一个唯一的逆元(如果 p 不是素数，这一点不成立。)
交换律、结合律、分配律在其上均成立。
- 伽罗瓦域上的运算大量地用于密码方面。整个数论运算都可在其中进行：它给数一个限制范围，除法将不会有任何舍入错误。许多密码系统都基于 $GF(p)$ ，这里 p 是一个大的素数。



- 如果一个多项式在指定的域上不能分解，则称为不可约多项式。
- 为了使算法更复杂，密码设计者也使用模 n 次不可约多项式的算术运算，它的系数是模 q 的整数， q 是素数。这些域称为 $GF(q^n)$ 。所有的运算都模 $p(x)$ ， $p(x)$ 是 n 次不可约多项式。
- $GF(2^n)$ 中的计算能用线性反馈移位寄存器以硬件快速实现。由于这个因素， $GF(2^n)$ 中的计算通常比 $GF(p)$ 上的运算更快，且计算 $GF(2^n)$ 上的幂要有效得多，这也用于计算离散对数。



- 对一个数进行因子分解就是找出它的素因子。例如 $10=2\times 5$
- 在数论中，因子分解是一个最古老的问题。分解一个数很简单，但是过程费时。
- **数域筛选法(NFS)**。对大于110位左右数字长的数来说，一般的数域筛选是目前已知的最快因子分解方法。当它最初提出来时，它是不实用的，但随着过去几年的一系列改进，这一点已经改变。
- 如果 n 是要被分解的数，那么NFS法渐进的时间估计值为

$$e^{(1.923+o(1))(\ln(n))^{1/3}} (\ln(\ln(n)))^{2/3}$$



- 密码学需要素数，任何合理规模的网络也需要这样的素数。
- Rabin-Miller算法：这是一个非常重要且常用的**大素数判定算法**。
- 首先选择一个待测的随机数 p ，计算 b ， b 是2整除 $p - 1$ 的次数(即 2^b 是能整除 $p - 1$ 的2的最大幂数)。然后计算 m ，使得 $n = 1 + 2^b m$ 。然后进行以下步骤：
 - (1) 选择一个小于 p 随机数 a 。
 - (2) 设 $j=0$ 且 $z=a^m \bmod p$ 。
 - (3) 如果 $z=1$ 或 $z=p-1$ ，那么 p 通过测试， p 可能是素数。
 - (4) 如果 $j>0$ 且 $z=1$ ，那么 p 不是素数。
 - (5) 设 $j=j+1$ 。如果 $j<6$ 且 $z \neq p-1$ ，设 $z=z^2 \bmod p$ ，然后回到第 (4) 步。如果 $z=p-1$ ，那么 p 通过测试， p 可能是素数。
 - (6) 如果 $j=b$ 且 $z \neq p-1$ ，那么 p 不是素数。



- 模指数运算是频繁地用于密码学的一种单向函数。计算 $a^x \bmod n$ 很容易。模指数运算的逆问题是找出一个数的离散对数，这是一个难题：求解 x ，使得 $a^x \equiv b \bmod n$ 。
- 例如：如果 $3^x \equiv 15 \bmod 17$ ，那么 $x = 6$ 。
- 不是所有的离散对数都有解(且只有整数才是合法的解)。对1024位的数求离散对数更加困难。
- 密码设计者对下面三个主要群的离散对数很感兴趣：
 - 素数域的乘法群： $GF(p)$ 。
 - 特征为2的有限域上的乘法群： $GF(2^n)$
 - 有限域 F 上的椭圆曲线群： $EC(F)$



- 许多公开密钥的算法的安全性是基于寻找离散对数的，因此对这个问题进行了广泛的研究。
- 如果 p 是模数且为素数，那么在 $GF(p)$ 上寻找离散对数的复杂性实质上与对同样大小的一个整数 n 进行因子分解的复杂性一样， n 是两个大致等长的素数的成绩，即 $e^{(1+o(1))(\ln(p))^{\frac{1}{2}}(\ln(\ln(p)))^{\frac{1}{2}}}$
- NFS更快一些，其渐进的时间估计值为： $e^{(1+o(1))(\ln(p))^{\frac{1}{3}}(\ln(\ln(p)))^{\frac{2}{3}}}$
- 计算离散对数与因子分解有紧密的关系。如果能解决离散对数问题，那么就能解决因子分解问题（逆命题是否正确还未证明）。
- 目前，在素数域上有三种方法计算离散对数：线性筛选法、高斯整数法和NFS。



2. 计算离散对数为什么困难以及困难度如何？



1. 在使用AES加密时，选择初始化向量（IV）非常重要。请解释为何CBC模式需要IV，以及如何选择安全的IV。
2. 用欧几里得算法计算 $\gcd(1024, 888)$

谢谢大家
